

DEVOPS TEN-MARK QUESTIONS WITH ANSWERS

Below are Questions 15–19 in full, followed by their respective easy, exam-ready answers. The answers use simple language, examples, headings and conclusions to help you prepare for your exam.

Question 15 – DevOps Implementation for a Healthcare Application

FULL QUESTION:

A healthcare application is being deployed across development, testing and production environments. The operations team needs consistent system configurations, isolated development environments and reliable application deployment. Develop a suitable DevOps implementation by applying an appropriate configuration management tool such as Ansible, Puppet, Chef or SaltStack, comparing its execution model with another configuration management approach, creating an isolated development environment using Vagrant, packaging the application using Docker, and applying suitable automation practices for deployment.

ANSWER

1. INTRODUCTION

DevOps combines software development and IT operations to deliver software quickly and reliably. In healthcare applications, it helps maintain consistent environments and protect sensitive patient information.

2. CONFIGURATION MANAGEMENT

Configuration management maintains consistent software settings across development, testing and production.

- Ansible: Uses YAML playbooks to automate configuration tasks.
- Puppet: Uses manifests and commonly follows a master-agent architecture.
- Chef: Uses recipes and cookbooks to define configurations.
- SaltStack: Supports configuration management and remote execution.

Example: All servers use the same application version and required software settings.

3. ANSIBLE VS. PUPPET

Ansible	Puppet
Usually agentless	Commonly uses agents
Uses YAML playbooks	Uses manifests
Often connects through SSH	Agents communicate with the Puppet server
Tasks commonly run when executed	Agents can periodically check configurations
Simple task-based automation	Useful for continuous configuration enforcement

4. VAGRANT FOR DEVELOPMENT ENVIRONMENTS

Vagrant creates and manages reproducible development environments.

- Creates virtual machines with predefined configurations.
- Provides similar environments to all developers.
- Reduces environment-related errors.

Example: Every developer uses a virtual machine with the same operating system and software.

5. DOCKER FOR APPLICATION PACKAGING

Docker packages an application and its dependencies into a container.

- Containers provide isolation.
- Applications can run consistently across compatible environments.
- Deployment becomes easier and more portable.

Example: The healthcare application and its required libraries are packaged into a Docker image.

6. DEPLOYMENT AUTOMATION

The deployment process can be automated as follows:

1. Developers commit code to a shared repository.
2. The CI pipeline builds the application.
3. Automated tests check the code.
4. Docker packages the application.
5. The application is deployed to testing.
6. After successful verification and approval, it is deployed to production.
7. Monitoring checks application health.

7. SECURITY AND RELIABILITY

- Encrypt patient data during transmission and storage.
- Apply access controls to patient records.
- Store passwords and secret keys securely.
- Test updates before production deployment.
- Maintain backups and monitor errors.

8. CONCLUSION

Configuration management, Vagrant, Docker and deployment automation provide consistent environments, reliable releases and better protection of healthcare data.

Memory trick: Configuration → Vagrant → Docker → Automation → Security.

Question 16 – Technical Checks in Continuous Integration

FULL QUESTION:

Explain the technical checks performed in a Continuous Integration workflow before accepting a build for further delivery.

ANSWER

1. INTRODUCTION

Continuous Integration (CI) is a software development practice in which developers frequently integrate code into a shared repository. Automated builds and tests verify the changes before further delivery.

2. CI WORKFLOW

The main stages are:

Code Commit → Static Analysis → Build → Testing → Quality and Security Checks → Build Approval for Further Delivery

3. TECHNICAL CHECKS

1. CODE COMPILATION AND BUILD

The source code is compiled or packaged to check whether the application can be built successfully.

Example: A Java application must compile without errors.

2. STATIC CODE ANALYSIS

Tools inspect source code without executing it to identify potential errors and coding-standard violations.

Example: Detecting unused variables or suspicious code.

3. UNIT TESTING

Individual functions or components are tested independently.

Example: Testing whether a payment function calculates the correct total.

4. INTEGRATION TESTING

Checks whether different modules work together correctly.

Example: Verifying that the login module communicates properly with the database.

5. CODE QUALITY CHECKS

These checks examine code duplication, maintainability and other quality issues.

6. SECURITY CHECKS

Automated tools identify known vulnerable dependencies, insecure coding patterns and exposed secrets.

7. TEST COVERAGE

Coverage reports indicate how much of the source code is exercised by automated tests. High coverage does not necessarily guarantee correct software.

8. BUILD RESULT AND REPORTING

The CI system reports success or failure. Failed builds are investigated and corrected before further delivery.

4. BENEFITS OF CI

- Detects defects early.
- Reduces integration conflicts.
- Improves software quality.
- Provides rapid feedback.
- Makes delivery more reliable.

5. CONCLUSION

CI automatically builds and checks integrated code before further delivery. It helps developers identify errors early and maintain software quality.

Memory trick: Build → Static Analysis → Unit Test → Integration Test → Quality → Security → Report.

Question 17 – DevOps for an IoT-Based Smart Manufacturing System

FULL QUESTION:

An IoT-based smart manufacturing system generates large volumes of application logs, device metrics and issue reports. The development and operations teams require a systematic approach for monitoring the platform and tracking software issues. Apply suitable DevOps tools and frameworks to manage the system by incorporating the following.

- a) Configure application and infrastructure monitoring using Nagios, Munin, Ganglia or Graphite.
- b) Implement centralized log handling using the ELK Stack.
- c) Select a suitable issue tracker such as Bugzilla, Trac, Redmine, GitLab or Jira.

d) Establish a mechanism for associating detected issues with development activities.

e) Integrate the monitoring and issue-tracking process into an Azure DevOps, Kubernetes or cloud-based IoT workflow.

ANSWER

1. INTRODUCTION

An IoT-based smart manufacturing system contains connected devices and sensors that generate large amounts of data. DevOps tools help monitor device health, analyze logs and track software problems.

2. (A) APPLICATION AND INFRASTRUCTURE MONITORING

Monitoring tools collect information about system performance and availability.

Tool	Purpose
Nagios	Monitors server availability and services
Munin	Collects and graphs system performance statistics
Ganglia	Monitors clusters and distributed systems
Graphite	Stores and visualizes time-series metrics

Important metrics include CPU usage, memory, response time, disk space, network traffic and device availability.

Example: Nagios alerts the operations team when a manufacturing server becomes unavailable. Graphite displays CPU usage over time.

3. (B) CENTRALIZED LOGGING USING ELK STACK

The ELK Stack consists of three components.

- Elasticsearch: Stores and searches log data.
- Logstash: Collects, processes and transforms logs.
- Kibana: Displays logs and creates dashboards.

Working:

1. Applications and IoT devices generate logs.
2. Logstash collects and processes the logs.
3. Elasticsearch stores and indexes them.
4. Kibana displays the logs for analysis.
5. Developers identify errors and investigate their causes.

Example: If a sensor fails repeatedly, developers search its error logs using Kibana.

4. (C) SELECTING AN ISSUE TRACKER

Issue trackers help manage bugs, tasks and development activities.

Tool	Purpose
Bugzilla	Bug tracking
Trac	Issue tracking and project management
Redmine	Task and project management
GitLab	Issue tracking integrated with source code and CI/CD
Jira	Issue tracking and agile project management

Suitable choice: GitLab or Jira can assign bugs to developers and track their progress.

5. (D) ASSOCIATING ISSUES WITH DEVELOPMENT ACTIVITIES

The following process connects detected problems with corrective actions.

1. Monitoring detects a failure.
2. Logs help identify the cause.
3. An issue is created in GitLab or Jira.
4. The issue is assigned to a developer.
5. The developer fixes the code and links the commit or merge request to the issue.
6. Automated tests verify the fix.
7. The issue is closed after successful verification.

Example: A sensor communication failure creates an issue, which is linked to the code change fixing the problem.

6. (E) INTEGRATION WITH A DEVOPS WORKFLOW

Suitable platforms include Azure DevOps, Kubernetes and cloud-based IoT platforms.

- Azure DevOps: Manages code repositories, pipelines, tests and work items.
- Kubernetes: Deploys and manages containerized applications.
- Cloud IoT platforms: Collect device data and support remote monitoring.

The workflow is:

IoT Devices → Monitoring and Logs → Issue Tracker → Code Fix → CI/CD Pipeline → Deployment → Monitoring

7. BENEFITS

- Detects failures quickly.
- Helps identify root causes.
- Improves developer and operations team coordination.
- Tracks bugs from discovery to resolution.
- Reduces manufacturing downtime.

8. CONCLUSION

Combining monitoring tools, ELK Stack, issue tracking and DevOps platforms provides centralized visibility and a systematic method for resolving IoT system problems.

Memory trick: Monitor → Logs → Issue → Fix → Deploy.

Question 18 – DevOps Workflow for a Cloud-Based E-Commerce Application

FULL QUESTION:

A software organization is developing a cloud-based e-commerce application that requires frequent code integration, automated testing, deployment and monitoring. The development team experiences build failures, configuration inconsistencies, deployment delays and difficulty in identifying application issues. Apply suitable DevOps tools and practices to establish an effective development and operations workflow.

- Implement a suitable configuration management approach using Ansible, Puppet, Chef or SaltStack for maintaining consistent application environments.
- Demonstrate the use of Docker for creating a portable and isolated deployment environment.
- Formulate a suitable Continuous Integration / Continuous Delivery workflow covering code integration, automated testing and deployment.
- Recommend suitable monitoring and log-handling tools for detecting application failures and analyzing system activities.

ANSWER

1. INTRODUCTION

A cloud-based e-commerce application requires frequent updates, reliable deployment and continuous monitoring. DevOps combines automation, testing and collaboration to deliver software safely and efficiently.

2. (A) CONFIGURATION MANAGEMENT

Configuration management maintains consistent settings across development, testing and production.

- Ansible: Uses YAML playbooks and is commonly agentless.
- Puppet: Uses manifests and commonly follows a master-agent model.
- Chef: Uses recipes and cookbooks.
- SaltStack: Supports remote execution and configuration management.

Example: All application servers use the same runtime version and configuration.

3. (B) DOCKER FOR PORTABLE DEPLOYMENT

Docker packages an application and its dependencies into a container.

Steps:

1. Write a Dockerfile.
2. Define the application runtime and dependencies.
3. Build a Docker image.
4. Run the image as a container.
5. Test the container.

6. DEPLOY IT TO THE CLOUD ENVIRONMENT.

Example: The same e-commerce container runs in development, testing and production.

Benefits: Portability, isolation, consistent environments and simpler deployment.

4. (C) CI/CD WORKFLOW

CI/CD automates the integration, testing and delivery of software.

Workflow:

1. Developers commit code to a shared repository.
2. The CI pipeline starts automatically.
3. The application is built.
4. Automated tests are executed.
5. Quality and security checks are performed.
6. The application is packaged into a Docker image.
7. The application is deployed to staging and verified.
8. After approval, it is deployed to production.

9. MONITORING CHECKS APPLICATION HEALTH.

Continuous Integration (CI) focuses on frequent code integration, builds and automated tests.

Continuous Delivery (CD) keeps validated software ready for release, with production deployment potentially requiring approval.

5. (D) MONITORING TOOLS

Monitoring detects application failures and performance problems.

- Nagios: Checks server and service availability.
- Graphite: Stores and visualizes performance metrics.
- Munin: Graphs system resource usage.

Important metrics include CPU usage, memory, response time, request rate and error rate.

Example: Monitoring detects increased response times during a major online sale.

6. LOG-HANDLING TOOLS

The ELK Stack centralizes and analyzes application logs.

- Logstash: Collects and processes logs.
- Elasticsearch: Indexes and stores logs.
- Kibana: Displays logs and dashboards.

Example: Developers use Kibana to identify a database error causing failed customer orders.

7. SOLVING COMMON PROBLEMS

Problem	Solution
Build failures	Automated builds and early feedback
Configuration differences	Ansible, Puppet or another configuration tool
Deployment delays	CI/CD automation
Application errors	Centralized logging
Performance issues	Monitoring and alerts
Failed releases	Rollback plans and health checks

8. CONCLUSION

Configuration management, Docker, CI/CD, monitoring and centralized logging create a consistent, automated and reliable workflow for e-commerce software.

Memory trick: Configuration → Docker → CI/CD → Monitoring → Logging.

Question 19 – CI/CD Workflow for a Financial Application

FULL QUESTION:

A financial application development team follows Continuous Integration and Continuous Delivery for releasing frequent software updates. Developers report build failures, integration conflicts and incomplete test coverage before deployment. Examine the CI/CD workflow and address the following.

- Analyze the stages involved in integrating source code changes.
- Verify the role of automated builds and testing in identifying integration defects.
- Evaluate the conditions required for promoting a successful build towards delivery.
- Inspect suitable monitoring and feedback mechanisms for identifying failures after deployment.
- Propose corrective measures for improving the reliability of the delivery workflow.

ANSWER

1. INTRODUCTION

A financial application must process transactions accurately and securely. CI/CD helps developers release frequent updates while detecting defects before deployment.

2. (A) STAGES OF SOURCE-CODE INTEGRATION

1. Code commit: Developers submit changes to a shared repository.
2. Version control: Git tracks changes and supports collaboration.
3. Integration trigger: A new commit starts the CI pipeline.
4. Code review: Changes are reviewed before merging.
5. Build: The application is compiled or packaged.
6. Testing: Automated tests check functionality and integration.
7. Quality and security checks: Tools examine code quality and known vulnerabilities.
8. Delivery: Successful builds move to staging and then toward production.

Example: A developer changes the money-transfer function, and the pipeline checks the change before it reaches customers.

3. (B) ROLE OF AUTOMATED BUILDS AND TESTING

Automated builds:

- Detect compilation and packaging errors.
- Ensure the application can be produced consistently.
- Reduce manual effort.

Automated testing:

- Unit testing: Checks individual functions.
- Integration testing: Checks communication between modules.
- Functional testing: Checks whether requirements work correctly.
- Security testing: Identifies potential security weaknesses.
- Regression testing: Ensures existing features continue working after changes.

Example: A test checks that transferring ₹1,000 reduces the sender's balance and increases the receiver's balance correctly.

4. (C) CONDITIONS FOR PROMOTING A SUCCESSFUL BUILD

A build should proceed only when the required quality checks are satisfied.

- The build completes successfully.
- Required automated tests pass.
- No unacceptable critical defects remain.
- Code-quality and security checks pass.
- Dependencies and configurations are validated.
- The build artifact is versioned and stored reliably.

- Staging tests and required approvals are completed before production release.

Example: If a payment test fails, the pipeline stops the release until the defect is fixed and the required checks pass.

5. (D) MONITORING AND FEEDBACK AFTER DEPLOYMENT

Monitoring verifies that the application continues to work correctly after release.

- Nagios: Detects service availability problems.
- Graphite: Displays response time and resource usage.
- ELK Stack: Collects and analyzes application logs.
- Alerts: Notify the team when defined thresholds are exceeded.
- User feedback: Helps identify problems experienced by customers.

Example: If transaction failures increase after an update, monitoring alerts the team and logs help identify the cause.

6. (E) CORRECTIVE MEASURES

Problem	Corrective measure
Build failures	Fix compilation errors and improve build checks
Integration conflicts	Integrate code frequently and communicate changes
Incomplete test coverage	Add tests for untested critical paths
Repeated defects	Perform root-cause analysis and add regression tests
Security vulnerabilities	Scan dependencies and fix vulnerable components
Deployment failures	Use staged rollout, health checks and rollback
Delayed error detection	Improve monitoring, alerts and centralized logging

7. RECOMMENDED WORKFLOW

Commit → Review → Build → Test → Quality/Security Checks → Staging → Approval → Production → Monitor → Feedback and Correction

For a financial application, sensitive data must be protected, access controlled and transactions validated throughout the workflow.

8. CONCLUSION

An effective CI/CD workflow identifies defects early, enforces quality checks before delivery and uses monitoring and feedback to improve reliability after deployment.

Memory trick: Commit → Build → Test → Quality → Deploy → Monitor → Feedback.

DEVOPS TWO-MARK QUESTIONS WITH ANSWERS

DEVOPS – ALL 14 FOUR-MARK ANSWERS

These answers use a storytelling flow + actual technical points + simple examples. Each answer is structured so you can understand the concept and write it in the exam.

Q1. Recall the configuration management tool that uses a master-agent architecture for managing infrastructure.

Answer: Puppet

Imagine a company has 100 servers. Configuring every server manually takes time and may cause errors. To solve this, the company uses Puppet, a configuration management tool.

Puppet commonly follows a master-agent architecture:

1. Puppet Master: Stores configuration rules and manages configurations.
2. Puppet Agent: Runs on managed servers and communicates with the master.
3. Manifest: Defines the desired configuration, such as installed packages and services.
4. Configuration enforcement: The agent checks and applies the required configuration.

Example: If a server needs Nginx installed, Puppet can ensure that Nginx is installed and configured correctly.

Conclusion: Puppet automates infrastructure configuration and helps maintain consistency across multiple servers.

Remember: Master → Agent → Manifest → Configuration.

Q2. Recognize the automation tool that uses playbooks to define configuration and deployment tasks.

Answer: Ansible

Imagine a developer needs to install software on several servers. Instead of performing the same steps manually, the developer writes an Ansible playbook.

1. Ansible uses YAML playbooks to define automation tasks.
2. It commonly connects to Linux servers through SSH.

3. Tasks can install packages, copy files and start services.

4. Ansible is usually agentless, so managed machines generally do not need a separate Ansible agent.

Example: An Ansible playbook can install Nginx, copy a website's files and start the web server automatically.

Conclusion: Ansible simplifies configuration management and deployment through reusable, automated playbooks.

Remember: Ansible = YAML + Playbooks + Automation.

Q3. Select the technology used to package an application and its dependencies into lightweight, isolated containers.

Answer: Docker

Imagine a web application works on a developer's laptop but fails on the production server because a library is missing. Docker helps prevent this problem by packaging the application and its dependencies into a container image.

1. A Dockerfile defines the application environment and build instructions.
2. Docker builds the instructions into an image.
3. The image is used to create a container.
4. The container provides an isolated environment for running the application.

Example: A Python web application and its required libraries can be packaged into a Docker image and run in development and production.

Conclusion: Docker provides portable, lightweight application packaging and helps maintain consistent environments.

Remember: Dockerfile → Image → Container.

Q4. Identify the tool commonly used for collecting and visualizing time-series metrics in a monitoring environment.

Answer: Graphite

Imagine an online shopping website becomes slow during a sale. The operations team needs to know how CPU usage, memory and response time change over time. Graphite helps store and visualize time-series metrics.

1. Applications or monitoring agents collect performance metrics.
2. Metrics are sent to Graphite's storage system.
3. Graphite stores values associated with timestamps.

4. Graphs help the team identify performance changes and trends.

Example: A Graphite graph may show CPU usage increasing from 40% to 95% when customer traffic rises.

Conclusion: Graphite helps visualize historical performance metrics and identify possible performance bottlenecks.

Remember: Collect metrics → Store data → View graphs → Find problems.

Q5. Choose the tool associated with centralized collection, processing and visualization of application logs.

Answer: ELK Stack

Imagine an application runs on ten servers. Each server generates logs, making it difficult to investigate an error by checking every machine separately. The ELK Stack centralizes log handling.

It consists of three main components:

1. Elasticsearch: Indexes, stores and searches log data.
2. Logstash: Collects, processes and transforms logs.
3. Kibana: Displays logs through dashboards and visualizations.

Example: If a payment fails, developers can search the centralized logs in Kibana to investigate the error.

Conclusion: The ELK Stack helps teams collect, analyze and visualize logs from multiple applications and servers.

Remember: Logstash → Elasticsearch → Kibana.

Q6. Continuous Integration involves automatically building and which code changes after they are integrated into a shared repository?

Answer: Testing

Continuous Integration (CI) is a practice in which developers frequently integrate code into a shared repository. Automated builds and tests check whether the integrated changes work correctly.

1. Developers commit code to a shared repository.
2. The CI pipeline starts automatically.
3. The application is built to detect compilation or packaging errors.
4. Automated tests verify the code and help identify defects.

Example: When a developer changes a login function, the CI system builds the application and runs tests to verify that login still works.

Conclusion: CI automatically builds and tests integrated code, allowing developers to detect errors early.

Remember: Commit → Build → Test → Feedback.

Q7. Which stack is commonly used for centralized log collection, processing and visualization?

Answer: ELK Stack

The ELK Stack is a collection of tools used to manage logs from multiple applications and servers in one place.

1. E – Elasticsearch: Stores and searches indexed logs.
2. L – Logstash: Collects and processes logs.
3. K – Kibana: Visualizes logs using dashboards.

Example: Logs from a web server, database and payment service can be analyzed centrally to find the reason for a failed transaction.

Conclusion: ELK makes log analysis easier by providing centralized storage, search and visualization.

Remember: E = Search and Store, L = Collect and Process, K = Visualize.

Q8. Which tool uses playbooks to automate configuration and deployment tasks?

Answer: Ansible

Ansible is an automation tool that uses playbooks written in YAML to describe tasks that should be performed on managed machines.

1. The developer creates a playbook.
2. The playbook defines tasks such as installing packages or copying files.
3. Ansible connects to managed machines, commonly through SSH.
4. It executes the tasks and reports the results.

Example: An Ansible playbook can install Python, copy application files and restart a web service across several servers.

Conclusion: Ansible reduces repetitive manual work and automates configuration and deployment.

Remember: Write playbook → Run Ansible → Configure servers.

Q9. Recall the DevOps platform that provides a workflow for planning, developing, testing and deploying applications.

Answer: Azure DevOps

Imagine a software team needs one platform to manage tasks, store code, run automated tests and deploy its application. Azure DevOps provides services that support this software development workflow.

1. Azure Boards: Plans work and tracks tasks and bugs.
2. Azure Repos: Stores and manages source code.

3. Azure Pipelines: Automates builds, tests and deployments.

4. Azure Test Plans: Supports test planning and management.

5. AZURE ARTIFACTS: HOSTS AND SHARES PACKAGES.

Example: Developers store code in Azure Repos, and Azure Pipelines automatically builds and tests it before deployment.

Conclusion: Azure DevOps integrates planning, coding, testing and delivery into a coordinated workflow.

Remember: Boards → Repos → Pipelines → Test Plans → Artifacts.

Q10. Recognize the issue tracker commonly used to manage tasks, bugs and project activities in software development workflows.

Answer: Jira

Imagine a software team discovers a login bug. The team needs to record the problem, assign it to a developer and track it until it is fixed. Jira is an issue-tracking and project-management tool that supports this process.

1. A team member creates an issue describing the problem.
2. The issue is assigned to a responsible developer.
3. Its status is updated as work progresses.
4. The issue is tested and closed after resolution.

Example: A bug named “Login fails with an incorrect error message” can be assigned to a developer and tracked from creation to completion.

Conclusion: Jira helps teams organize tasks, manage bugs and monitor project progress.

Remember: Create → Assign → Track → Resolve.

Q11. Summarize the role of Puppet agents in managing configurations under a Puppet master.

ANSWER

Imagine a company uses Puppet to manage hundreds of servers. The Puppet Master defines the desired configurations, while Puppet Agents run on managed machines and help enforce those configurations.

1. The Puppet Master stores configuration rules and manifests.
2. The Puppet Agent communicates with the master to obtain its configuration information.
3. The agent checks the machine's current state against the desired state.
4. The agent applies required changes and helps maintain configuration consistency.

Example: If a server is missing a required package, the Puppet Agent can install it according to the defined configuration.

Conclusion: Puppet Agents automate configuration management, reduce manual administration and help keep servers consistent.

Remember: Master defines → Agent checks → Agent applies → Server stays consistent.

Q12. Differentiate between Ansible and Puppet based on their execution and configuration management approaches.

ANSWER

Imagine an organization needs to configure multiple servers. It can use either Ansible or Puppet, but they manage tasks differently.

Ansible	Puppet
Usually agentless	Commonly uses agents
Uses YAML playbooks	Uses manifests
Commonly connects through SSH	Agents communicate with the Puppet server
Tasks often run when the playbook is executed	Agents can periodically check and enforce configurations
Well suited to task-based automation	Well suited to maintaining a desired configuration state

Example: Ansible can run a playbook to install Nginx on several servers. Puppet can continuously help ensure that Nginx remains installed and configured according to policy.

Conclusion: Ansible emphasizes playbook-based task execution, while Puppet commonly emphasizes agent-based configuration enforcement.

Remember: Ansible = Playbooks; Puppet = Master, Agents and Manifests.

Q13. Demonstrate the use of Docker for creating an isolated environment for deploying a web application.

ANSWER

Imagine a developer creates a Python web application that requires specific libraries. Installing those libraries separately on every server can cause dependency conflicts. Docker packages the application and its dependencies into a container image.

STEPS

1. Create the application: Write the web application and its dependency file.
2. Create a Dockerfile: Define the base image, dependencies and startup command.

3. Build an image: Use `docker build` to package the application.
4. Run a container: Use `docker run` to start the application in an isolated environment.
5. Access the application: Map a host port to the container port if external access is needed.
6. Deploy the application: Use the same image in compatible testing and production environments.

EXAMPLE DOCKERFILE

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

Example commands:

```
docker build -t my-web-app .
docker run -p 5000:5000 my-web-app
```

Explanation: The Dockerfile defines the environment, `docker build` creates the image, and `docker run` starts a container. The application must listen on the appropriate port for the port mapping to work.

Conclusion: Docker provides an isolated and portable deployment environment and reduces dependency-related problems.

Remember: Write Dockerfile → Build image → Run container → Access application.

Q14. Formulate a suitable monitoring approach using Nagios or Graphite for identifying application performance issues.

ANSWER

Imagine an e-commerce application becomes slow whenever many customers access it. The operations team needs to detect service failures and investigate performance changes. Nagios and Graphite can be used together because they serve complementary purposes.

1. NAGIOS FOR AVAILABILITY MONITORING

Nagios checks whether servers, services and other monitored resources are available.

- Configure checks for server and application availability.
- Monitor service health and important resource thresholds.
- Generate alerts when a check fails.
- Notify the operations team so they can investigate.

Example: Nagios alerts the team when the web server stops responding.

2. GRAPHITE FOR PERFORMANCE MONITORING

Graphite stores and visualizes time-series metrics.

- Collect CPU usage, memory usage, response time and request rates.
- Store metrics with timestamps.
- Display graphs to identify changes over time.
- Compare normal and abnormal periods to locate possible bottlenecks.

Example: Graphite shows CPU usage rising to 95% whenever website traffic increases.

3. COMBINED MONITORING APPROACH

1. Nagios detects that the application is unavailable or a service check has failed.
2. The team checks Graphite graphs to examine recent performance trends.
3. Application logs can then help identify the root cause.
4. The team corrects the problem and verifies that the service returns to normal.

Conclusion: Nagios is useful for availability checks and alerting, while Graphite is useful for visualizing performance metrics. Together, they help detect and investigate application problems.

Remember: Nagios = Check and Alert; Graphite = Store and Graph.

FINAL ONE-LINE REVISION

Question	Answer to remember
Q1	Puppet – master-agent configuration management
Q2	Ansible – YAML playbooks
Q3	Docker – application containers
Q4	Graphite – time-series metrics
Q5	ELK Stack – centralized logs
Q6	Testing – CI automatically builds and tests code
Q7	ELK Stack – Elasticsearch, Logstash, Kibana
Q8	Ansible – playbook-based automation
Q9	Azure DevOps – integrated development workflow
Q10	Jira – issue and task tracking
Q11	Puppet Agent – checks and applies configuration
Q12	Ansible – playbooks; Puppet – master-agent
Q13	Dockerfile → Image → Container
Q14	Nagios – alerts; Graphite – performance graphs